A Multi-Objective Genetic Algorithm for Simulating Optimal Fights in StarCraft II

Jonas Schmitt
Friedrich-Alexander-Universität Erlangen-Nürnberg
Email: jonas.schmitt@fau.de

Harald Köstler
Friedrich-Alexander-Universität Erlangen-Nürnberg
Email: harald.koestler@fau.de

***Abstract*—The goal of this work is to develop a multi-objective genetic algorithm for simulating optimal fights between arbitrary units in the real-time strategy game StarCraft II. As there is no freely available application programming interface for controlling units in the game directly, this first requires an accurate simulation of the actual game mechanics. Next, based on the concept of artificial potential fields a general behavior model is developed which allows controlling units in an optimal way based on a number of real-valued parameters. The goal of each individual unit is to maximize their damage output while minimizing the amount of received damage. Finding parameter values that control the units of two opposing players in an optimal way with respect to these objectives can be formulated as a multi-objective continuous optimization problem. This problem is then solved by applying a genetic algorithm that optimizes the behavior of each unit of two opposing players in a competitive way. To evaluate the quality of a solution, only a finite number of solutions of the opponent can be used. Therefore, the current optima are repeatedly exchanged between both players and serve as input for the simulated encounter. By comparing the solutions of both players at the end of the optimization, it can be estimated if one of the two players has an advantage. Finally, in order to evaluate the effectiveness of the presented approach, a number of sample build orders, which correspond to the amount of units that have been produced until a certain point of time, serve as input for several optimization runs.**

I. Introduction

Electronic sports (abbreviated eSports), which is a term for organized video game competitions, continues to gain in popularity. With price pools comparable to and viewer numbers even surpassing those of traditional sport events[1], an increasing number of video games is designed with an adaption as professional gaming playground in mind. A prominent example is StarCraft® II: Heart of the Swarm®, a real-time strategy game from Blizzard Entertainment[2]. It is not only present on most important international eSport tournaments currently held, but has even evolved into a professional sport within South Korea. In a typical game two players face each other. Both opponents can choose among three races (Terran, Zerg and Protoss), which differ in their strengths and weaknesses. Starting under the same conditions, both players' goal is to produce the right composition of units, defeat the opponent's units in combat and ultimately win by destroying all of his structures. In general, a game can be divided into three different phases: Early, mid and late game. The early game mostly focuses on developing the economy and building up a decent army while occasionally fending off early aggression. In the mid game the majority of fights take place with both players continuously increasing the size of their armies. In case the game continues further without a clear winner, the end game phase is reached and both players are able to utilize their races' full potential, including the most powerful units and upgrades. In all three phases a player has to choose among numerous possibilities and strategies. The outcome of a game mainly depends on two aspects of a player's decision making: Macro- and micromanagement. The former describes the production order of structures and units and the latter how units are controlled in combat situations. To ensure that both opponents have equal winning chances, all three races must be balanced with respect to the aforementioned aspects in all phases of the game.

An overview about common techniques for optimizing AI behavior in real-time strategy games can be found in [1]. In [2] and [3] an approach for optimizing build orders, i.e., the order in which structures and units are produced by a player, is presented. These build orders then represent the potentially best macromanagement strategy a certain player can execute within a fixed amount of time. By simulating a battle between the units created within the respective build order, it can be predicted which of two opponents has an advantage at the current point of time in the game, assuming that all units are controlled optimally. Optimizing unit behavior in real-time strategy games based on parametrized artificial potential fields is a common approach (see for example [4] [5] [6] [7] [8]) that originally stems from robotics [9] [10]. In [11] it was shown that this approach is also suitable for StarCraft II, producing reasonable results for different types of units. However, only encounters between units of a single type were considered and the units' special abilities were completely ignored. Moreover, the simulation of unit behavior was based on simplified game mechanics and did not fully cover the actual in-game complexity of micromanagement. In this paper the shortcomings of this work are addressed and it is further extended to produce optimal combat behavior for heterogeneous unit groups in a reasonable accurate simulation of the StarCraft II games mechanics.

[1]According to the "Global Growth of eSports Report" by newzoo games market research from 2015

[2]©2013 Blizzard Entertainment, Inc. All rights reserved. StarCraft and Heart of the Swarm are trademarks or registered trademarks of Blizzard Entertainment, Inc., in the U.S. and/or other countries.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

II. Simulation

Until now there exists no freely available API (Application Programming Interface) for controlling units in StarCraft II. To ensure the relevance of eventual results, it is therefore necessary to simulate the game by accurately emulating the actual game mechanics. As this work focuses on the simulation and optimization of micromanagement, the following assumptions are made:

- Units can move and interact on a two-dimensional rectangular area that possesses neither obstacles nor height differences.
- Structures are ignored.
- No new units are created during the simulation. Only those occurring in a predefined build order are considered.

In StarCraft II each unit is characterized by a number of attributes each of which possesses a distinctive value. Additionally to these attributes units can have special characteristics and abilities, which are either unit or race specific. For example, all Zerg units are able to regenerate their health at a slow rate. Also, some attacks exhibit special characteristics, for instance, the attack of the Baneling, a Zerg unit, causes damage to all ground units within a circular area of influence, and after execution the Baneling is removed from the game. While characteristics are always active, special abilities either cost an additional resource called energy or can only be applied consecutively after a certain cooldown period has passed. An example is the Stalker's blink ability. If researched by the player, this unit is able to instantly teleport over a certain distance with respect to a cooldown between consecutive applications.

Despite these details, in principle a unit is able to perform the following actions at each arbitrary point of time in the game:

- **Attacking.** If the time span from the unit's last attack is smaller than its cooldown, it is able to attack exactly one unit within its range. The damage is computed according to the enemy's type and the applying bonuses. If present, special attack characteristics must be considered.
- **Moving.** If the unit decides not to or is not able to attack, it can move in each direction according to its speed.
- **Special Abilities.** If the unit possesses special abilities, these can be applied either additionally or instead of the aforementioned actions depending on the characteristics of the ability.

A. Potential Fields

To control units such that they behave optimally in combat, artificial potential fields are employed. In the context of a real-time strategy game bodies correspond to units and forces denote the direction each unit moves. Thus, by artificially creating different potential fields around a unit's position other units' interactions with it can be defined. The goal of each individual unit is to deal a maximum amount of damage while receiving a minimum amount at the same time. As a unit can only deal damage as long as an enemy stays within its attack range, an attractive potential is required that forces the unit to approach enemy units until this condition is fulfilled. On the contrary, a unit can only take damage as long as it stays within the enemy's range, therefore a repulsive potential is required that forces the unit to retreat. Finally, friendly units should not act on their own, but as a group instead, both approaching and retreating collectively. This is realized by adding a third potential which creates an artificial attraction between friendly units and to a certain degree forces them to follow their companions' paths instead. A unit's movement direction is then computed by accumulating the gradients of all potential fields evaluated at the respective position.

While the overall behavior is clear, each potential field comes with certain degrees of freedom, as it is unclear how the various unit attributes should contribute in an arbitrary situation. It is especially not obvious how the magnitudes of the resulting forces should compare to each other in case that multiple fields overlap at a position. In order to achieve optimal combat behavior it is necessary to parametrize the described potential fields, such that all relevant attributes can be weighted arbitrarily. Therefore, they are modeled as functions of the distance between both involved units which contain 14 additional parameters with a range of values of $[0, 1]$, as described in Figure 1, 2 and 3. Additional factors that occur within their computation are summarized in I. As the values of the coefficients that occur within the computation of the gradients of these functions differ significantly, they are mapped to the same uniform range.

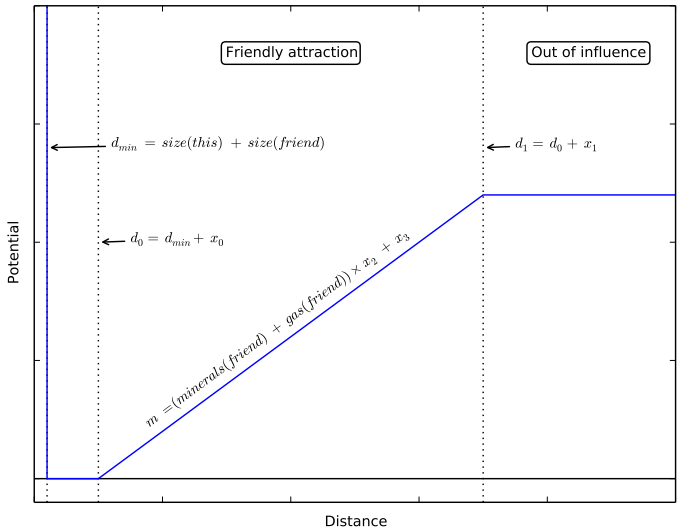


Fig. 1: Function describing the attractive potential of friendly units.

Figure 1 denotes the attractive potential of friendly units with a range that can be arbitrarily chosen according to two parameters. The magnitude of the gradient and thus the strength of the resulting attraction mainly depends on the amount of resources, i.e., minerals and vespene gas. required to produce the unit. The costlier a unit, the more important it usually is, making it necessary to accompany and protect it

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

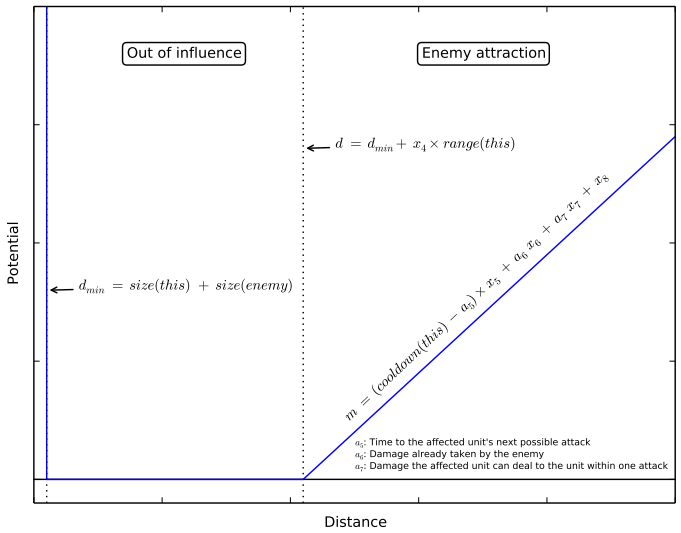


Fig. 2: Function describing the attractive potential of enemy units.

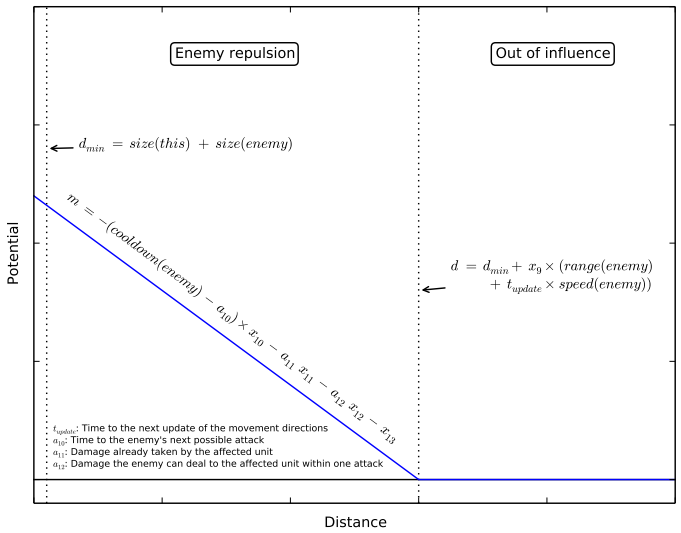


Fig. 3: Function describing the repulsive potential of enemy units.

TABLE I: Additional factors that occur within the potential field computation.

a_5	Time to the affected unit's next possible attack
a_6	Damage already taken by the enemy unit during the simulation
a_7	Damage the affected unit can deal to the enemy unit within one attack
t_{update}	Time to the next update of the movement directions
a_{10}	Time to the enemy unit's next possible attack
a_{11}	Damage already taken by the affected unit during the simulation
a_{12}	Damage the enemy can deal to the affected unit within one attack

with a number of cheaper units. Additionally, collisions, which happen when friendly units approach each other at a distance smaller than the sum of their two sizes, need to be avoided. This is achieved by setting the magnitude of the potential to a value near infinity, such that the two units are forced to move apart until the condition is fulfilled. Figure 2 denotes the attractive potential of enemy units. Its range depends on the attack range of the affected unit while the magnitude of the attraction is controlled by the following parameters:

- The less time is left until the affected unit is able to attack the higher the attraction, as approaching the enemy only makes sense when an attack is possible.
- The more damage the enemy has already taken the easier it is to kill and thus the higher is the attraction.
- The more damage can be applied to the enemy within one attack the higher the attraction.

Collisions between opposing units also need to be avoided which is accomplished in the same way as described above.

Finally, Figure 3 denotes the repulsive potential of enemy units. Its range depends on the attack range of the enemy unit, but as an additional safety buffer, the movement speed of the enemy is also taken into account. The magnitude of the repulsion is controlled by the same parameters applying to Figure 2, but from the perspective of the enemy unit. Besides all relevant unit attributes, the concrete behavior of the described potential fields entirely depends on the 14 parameters $x_0, \ldots, x_{13}$ that occur within their calculation.

B. Workflow

As a human player is only able to perform a limited number of actions per second, it is reasonable to allow making decisions only at discrete time steps. After fixing all relevant parameters, the outcome of an encounter between two groups of units is simulated in the following way. At the beginning the units of both players are placed at equidistant positions on opposite sides of the field. During a time step the following actions are performed by each unit:

- If attacking is possible, a target is chosen among all enemies within attack range by favoring units that can be defeated if the wasted damage[3] does not exceed a certain threshold, which is set by an additional parameter x_{14}. These are then prioritized by the amount of applicable damage.
- Else, the unit moves and its position at the next time step is computed with the following equation:

$$\vec{p_{i+1}} = \vec{p_i} + \frac{\vec{F}}{\|\vec{F}\|} \times s \tag{1}$$

where $\vec{p_i}$ is the position at time step i, $\vec{F}$ the current force and s the movement range within one time step[4].

The force of each unit is recomputed after a fixed number of time steps by accumulating the gradients of all potential fields applying to it:

$$\vec{F} = \sum_{j=1}^{n} \vec{F_j} \tag{2}$$

[3]The difference between the remaining health of the target and the damage the attack could theoretically deal to a unit with the same attributes.

[4]The movement range equals the product of the length of a time step and the speed of the unit.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

where n is the number of potential fields affecting the unit and $\vec{F}_j$ the gradient of the jth potential field. In case a unit exhibits special abilities, these need to modeled and considered during each time step, too. If their application incurs any kind of decision making, additional parameters are introduced[5]. The simulation finishes when either all units of a player have been defeated or its duration exceeds a certain limit. As described above, the behavior of each individual unit is controlled by the values of 15 real-valued parameters, where $x_0 \dots x_{13}$ describe the movement behavior and x_{14} sets the attack threshold in case of multiple targets, plus possibly additional parameters controlling the use of special abilities. In consequence, the values of these parameters completely describe the behavior of each individual unit.

What is left to be answered is how many sets of parameters should be assigned to the units of a build order. Compared to the overall amount of units in a build order the number of different units is small. Units of the same kind share attributes and special abilities, such as for example the same range and movement speed, which facilitates the synchronization of their behavior. Moreover, it can be expected that acting as a group increases their combat effectiveness because it enables them to combine their attacks and kill single enemy units faster. Therefore, only one set of parameter values is assigned to all units of the same kind[6]. This does not only prevent a linear growth of the number of parameters with the length of a build order, but also enables the optimization of the behavior of similar units in a consistent way, which is covered in the following section.

III. Optimization

A. Problem Formulation

The goal of this work is to predict which of two players wins in a combat based on the units produced and assuming optimal control. According to the last section, the behavior of a subgroup of units of the same kind is controlled by at least 15 parameters with a range of values of $[0, 1]$. The combat effectiveness of each individual unit is represented by two objectives: The overall damage the unit is able to cause to enemy units (f_d) and how much of its own health it is able to preserve (f_h). For the sake of simplicity, from now on the set of parameters that describes the combined behavior of all units of a player is denoted as his *strategy*. Under the assumption that the optimal strategy of the opponent is known, its effectiveness according to both objectives can be evaluated by simulating the respective encounter with both strategies as input. Calculating the overall success of a strategy with respect to an objective can be simply achieved by adding up the corresponding measure at the end of the simulated encounter. Not all units that possess the same amount health are also equally important. For example the sum of health of three Zerglings is slightly larger than that of one Infestor, although the latter unit is much more valuable, possessing powerful special abilities, which needs to be incorporated when evaluating both objectives. The cost (in minerals and vespene gas) of a unit represents a good approximation of its importance and is therefore used as weighting factor when evaluating the respective objective. Furthermore, to prevent units from being completely passive, an additional penalty is introduced: The health of units that have not attacked at least once during the simulation is not taken into account when evaluating the outcome. The effectiveness of a strategy with respect to the two objectives, f_d and f_h, is then calculated by the following formulas from the outcome of a simulated encounter with U as the set of units of the player whose strategy is evaluated and V as the set of units of his opponent:

$$f_d(U, V) = \sum_{v \in V} (minerals(v) + gas(v)) \times damage(v) \quad (3)$$

where $damage(v)$ is the sum of health and shield the unit v has lost, measured at the end of the simulation.

$$f_h(U, V) = \sum_{u \in U} (w(u) \times (minerals(u) + gas(u)) \times (health(u) + shield(u))) \quad (4)$$

where $health(u)$ is the remaining health and $shield(u)$ the remaining shield[7] of unit u, measured at the end of the simulation. The weighting function $w(u)$ is defined as follows:

$$w(u) = \begin{cases} 1 & \text{if } u \text{ has attacked} \\ 0 & \text{otherwise} \end{cases}$$

Maximizing f_d and f_h as output of a simulation with the strategy of the first player as input leads to a multi-objective continuous optimization problem. Because of the untraceable relationship between in- and output of a simulation, gradient-based methods are not applicable. As a remedy search heuristics which do not require such knowledge can be applied, such as genetic algorithm. To solve the problem of competitively optimizing the strategies of two players, a multi-objective genetic algorithm based on NSGA-II [12] is employed.

B. Genetic Algorithm

In the following the different parts of the algorithm are described in detail.

1) Fitness Evaluation: Finding optimal strategies for both players would theoretically require to evaluate all possible solutions against each other. Because the search space is continuous the number of possible solutions is uncountable. To avoid this dilemma and still get a reasonable fitness approximation, each solution is only evaluated against a limited number of the best solutions of the opponent. At the beginning, n additional solutions are generated randomly and exchanged

[5]Describing them all goes beyond the scope of this work but it is assured that all units occurring within the tested build orders are modeled completely, including all special abilities, parameterizing relevant behavioral aspects.

[6]For example in case of a build order containing four marines and two marauders, one set of parameters is assigned to the four marines and another one to the two marauders.

[7]For Terran and Zerg units the shield is always zero.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

between both populations. To evaluate an individual, these solutions act as second input for n simulations and objective function evaluations whose average then forms its fitness in both objectives:

$$f_d(X) = \frac{1}{n}\sum_{i=1}^{n} f_d(U_X, U_{Y_i}) \quad (5)$$

$$f_h(X) = \frac{1}{n}\sum_{i=1}^{n} f_h(U_X, U_{Y_i}) \quad (6)$$

$f_d(X)$ denotes the fitness of individual X in the first and $f_h(X)$ in the second objective. $Y_1, \ldots, Y_n$ are the n individuals currently used for fitness evaluation. U_X and U_{Y_i} denote the corresponding sets of units at the end of a simulation with X as input for the first and Y_i as input for the second player.

After a fixed number of generations both players exchange their n best solutions, which then replace the previous ones used for fitness evaluation. This scheme results in a competitive optimization of both populations. By repeatedly exchanging optima, both populations are forced to adapt to changing requirements, which prevents them from overfitting with respect to a subset of possible solutions of the opponent. After passing the last generation, the individuals of both population can finally be evaluated against each other to examine the effectiveness of the corresponding build orders in a direct comparison.

2) Crossover: The purpose of crossover is to produce improved solutions by combining the chromosome of existing ones, selected from the current population. In order to find the best crossover operator for the existing case, their performance is tested by competitively optimizing two sample build orders that include units with divers characteristics and abilities.

- **Player 1:** 4 Marines with Combat Shield and Stimpack, 2 Marauders with Stimpack
- **Player 2:** 2 Zealots with Charge, 2 Stalkers with Blink

The operators are tested in combination with binary tournament selection and the three different mutation operators mentioned in Table III, in two optimization runs over 500 iterations and with population sizes of 1600 for both opponents. Each individuals is evaluated against ten strategies. Every tenth iteration, the objective function is adapted by exchanging optima between both populations. As self-adaptive simulated binary crossover requires additional objective function evaluations to achieve comparable runtimes, the number of generations is reduced by 30 %. The results of the experiments are summarized in Table II. According to this comparison, self-adaptive simulated binary crossover provides superior performance, surpassing the other tested operators in all relevant metrics. It is therefore exclusively considered for the implementation of crossover within the presented algorithm, and its functioning is briefly described in the following.

[8]Online performance denotes the average of all objective function evaluations.

[9]Offline performance denotes the average of the optima of all generation.

[10]The number of crossover points is one less than the number of different units in the build order, but at least one.

TABLE II: Results for different crossover operators. Online and offline performance are averaged over both populations and all runs.

	Online Performance[8] (Damage, Health)	Offline Performance[9] (Damage, Health)	Number of Evaluations	Runtime
N-Point Crossover [13][10]	66.85, 41.61	90.64, 62.60	60048000	68 min
Uniform Crossover [14]	69.04, 37.67	89.88, 57.49	60048000	64 min
Intermediate Crossover [15]	51.53, 51.94	76.70, 73.21	60048000	61 min
Line Crossover [15]	52.24, 52.78	82.14, 72.50	60048000	60 min
SBX [16]	53.25, 51.27	84.02, 70.94	60048000	66 min
Self-Adaptive SBX [17]	76.90, 54.49	98.59, 80.58	61602960	62 min

3) Mutation: Mutation in the context of genetic algorithms means altering single parts of an individual's chromosome (genes) with the purpose of maintaining genetic diversity in the population. The chance of mutation is determined by the mutation probability. To decide which genes to mutate theoretically requires the creation of n random numbers for each individual where n is the length of the chromosome. In [18] different alternative schemes have been investigated. According to this comparison, the suggested mutation clock scheme possesses superior performance. In the context of real-coded genetic algorithms, different mutation operators for altering single genes have been proposed. To determine which one leads to the best performance in combination with adaptive SBX, three common mutation operators are tested in ten optimization runs with the same setting used in the crossover evaluation. The results can be seen in Table III. Overall,

TABLE III: Results for different mutation operators. Online and offline performance are averaged over both populations and all runs.

	Online Performance (Damage, Health)	Offline Performance (Damage, Health)	Number of Evaluations	Runtime
Uniform Mutation[11]	84.81, 52.51	99.17, 79.69	170890850	190 min
Polynomial Mutation [18]	79.28, 50.31	98.60, 77.50	166659150	184 min
Gaussian Mutation [18]	62.48, 62.68	94.66, 90.81	165756100	203 min

uniform mutation performs slightly better than polynomial mutation, but possesses similar characteristics. Gaussian mutation on the other hand leads to more balanced results with respect to both objectives while sacrificing online performance in the first objective. It is although the only mutation operator that is able to guide the search towards finding nearly optimal solutions in both objectives, with equally good offline performance and should therefore be preferred.

4) Main Loop: By combining the aforementioned components, the general structure of the genetic algorithm is summarized in Algorithm 1.

[11]The old value of the parameter is replaced by a uniformly distributed random value.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Main Loop

procedure MAIN(p, m, n, s)
  ▷ Initialize both players
  INITIALIZE($P^{(1)}$, p, $S^{(1)}$, s)
  INITIALIZE($P^{(2)}$, p, $S^{(2)}$, s)
  ▷ Run the genetic algorithm on both populations for $m \times n$ generations
  for $i = 0 \dots m$ **do**
    OPTIMIZE($P^{(1)}$, p, n, $S^{(1)}$, s, $\hat{S}^{(2)}$)
    OPTIMIZE($P^{(2)}$, p, n, $S^{(2)}$, s, $\hat{S}^{(1)}$)
    ▷ Exchange the s best individuals
    $S^{(1)} = \hat{S}^{(1)}$
    $S^{(2)} = \hat{S}^{(2)}$
    ▷ Reevaluate both populations
    EVALUATE-AND-SORT($P^{(1)}$, $S^{(1)}$)
    EVALUATE-AND-SORT($P^{(2)}$, $S^{(2)}$)

procedure INITIALIZE(P, p, S, s)
  ▷ randomly generate an initial population P with p individuals
  GENERATE-POPULATION(P, p)
  CHOOSE-STRATEGIES(S, s) ▷ randomly choose s evaluation strategies $S = \{S_1, \dots, S_s\}$
  EVALUATE-AND-SORT(P, S)

procedure EVALUATE-AND-SORT(P, S)
  EVALUATE(P, S) ▷ evaluate the individuals contained in P using S
  NON-DOMINATED-SORTING(P)
  CROWDING-DISTANCE-ASSIGNMENT(P)

procedure OPTIMIZE(P, p, n, S, s, $\hat{S}$)
  for $i = 1 \dots n$ **do**
    $P' =$ SELECT(P) ▷ select p individuals from P and save them in P'
    $Q =$ CROSSOVER(P') ▷ create p new individuals Q from P' using crossover
    MUTATE(Q) ▷ mutate a subset of the individuals contained in Q
    EVALUATE(Q, S)
    $P = Q \cup P$
    NON-DOMINATED-SORTING(P)
    CROWDING-DISTANCE-ASSIGNMENT(P)
    $P = P \setminus \{P_{p+1}, \dots, P_{2p}\}$ ▷ remove the worst individuals from P such that $|P| = p$
  $\hat{S} = P \setminus \{P_{s+1}, \dots, P_p\}$ ▷ store the s best individuals of P in $\hat{S}$

IV. Results and Discussion

To evaluate the presented approach, one sample build order is chosen for all three races which contains units that in sum cost the same amount of resources[12]:

- Zerg: 2 Zerglings, 5 Roaches.
- Terran: 5 Marines, 2 Marauders.
- Protoss: 2 Zealots, 2 Stalkers.

These build orders are then pairwise optimized in five test runs using the following settings: For both players a total population of 5000 is maintained, which is then optimized over 200 generations using ten individuals for fitness evaluation which are exchanged every tenth generation. For selection a tournament size of two is chosen and the mutation probability is set to $\frac{1}{n}$ where n is the length of the chromosome. To simulate the game with a reasonable accuracy, time steps with a length of 10 milliseconds are used, and the movement direction of each unit is updated every tenth time step. Zerg units possess an increased movement speed when they operate on creep. It is a special ground layer that is automatically created by Zerg buildings and some units and can be used tactically. To estimate the importance of creep placement, the Zerg build order is evaluated once with and once without assuming creep on the ground. After an optimization run is finished, the 100 best individuals of both players are evaluated in a direct competition, such that each individual is evaluated against all 100 of the other player. By computing the average of all test runs an estimate can be obtained if the build orders are balanced with respect to each other. If one build order is able to outperform the other one in all runs, it probably has an advantage, at least under the assumptions that have been made to simulate the game.

Table IV shows the results of all encounters. $\bar{f}_d$ is the average damage in percent that the units of the denoted player were able to cause to their enemies in the final comparisons and σ_d the standard deviation of all test runs. As the average health of the units of one player is correlated to the damage caused by the units of the other player, it is omitted in the table. Without creep the Zerg build order is inferior to the other two, as in all test runs its average damage output is lower than its enemy's. Although, the picture changes when creep is assumed. Now, in both encounters the difference between the averages of both players is relatively small and no clear winner can be determined. Thus, it can not be assumed that any of both opponents has a significant advantage. In general, it can be concluded that the closer the outcome of an encounter is, which corresponds to a small difference between the averages of both players, the lower is the standard deviation. In three of the investigated encounters a clear winner, who prevails in

TABLE IV: Results of the final comparison of the investigated encounters.

Encounter	$\bar{f}_d$ (Player 1)	σ_d (Player 1)	$\bar{f}_d$ (Player 2)	σ_d (Player 2)	Advantage
Zerg vs. Terran	28.90	3.96	92.48	4.68	Terran
Zerg vs. Terran (Creep)	67.42	6.17	79.86	11.87	None
Zerg vs. Protoss	29.47	7.13	71.40	1.97	Protoss
Zerg vs. Protoss (Creep)	64.64	14.28	61.64	7.38	None
Terran vs. Protoss	59.57	4.97	86.60	5.38	Protoss

[12]Minerals and vespene gas are weighted equally.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

all test runs over his opponent, could be determined. In the other two cases there are a number of factors that could have prevented the algorithm from reaching the same final state in all test runs. First of all, it is possible that one or multiple parameters of the algorithm have been chosen in an insufficient way. Increasing the population size, number of generations, fitness evaluations per individual or including more individuals into the final comparison could possibly lead to the desired stability. If none of the above factors leads to an improvement, an adaption of the algorithm must to be considered.

A. Evaluation of Unit Behavior

Finally, it is examined if the algorithm is able to produce unit behavior that can also be considered as reasonable from an outer perspective. For this purpose, a fight between the two solutions of each encounter that perform best in the final comparison is simulated and the paths of all units are tracked. The resulting plots can be seen in Figure 4 to 6.

First of all, compared to the preceding work [11] the resulting paths are significantly more fine-grained. The improved accuracy of the simulation allows a more precise control of the individual units, which leads to a higher number of changes in the movement directions because each unit tries to outmaneuver its enemies. In general, the resulting paths are of complicated nature and can not be easily described. All units have in common that they continuously try to stay in the sinks of the accumulated potentials, which corresponds to the area where they are able to attack while remaining unharmed by their enemies. This results in a competitive back and forth movement between opposing units, and is represented by the zigzag patterns occurring in all of the visualized encounters.

Moreover, there are a number of recurring patterns that evidently correspond to behavior that can be considered as intelligent. As it was explained at the end of Section II, the formation of collective behavior is facilitated by parameter sharing. Therefore, as expected, the units do not act as sole individuals but instead as a group which often results in the connection of their paths. Especially units of the same kind usually form subgroups, for example, the blue paths of the Marines in Figure 4. As they are able to move with the same speed and attack at the same range, it is easy for them to synchronize their behavior in an effective way. For example melee units such as Zealots all share the goal of approaching the enemy as close as possible, because otherwise they are unable to attack. On the other hand, ranged units try to position themselves out of their enemies range, such that they are able to attack but stay unharmed at the same time. By acting as a group units are able to exploit these similarities and combine their combat strength in a synchronous way. Another evident behavior is that faster units effectively utilize their speed advantage. For example in the encounter between Protoss and Terran, denoted by Figure 6, the Stalkers (dark blue) constantly avoid the lower ranged Marines (red), which is reflected in their large area of movement.

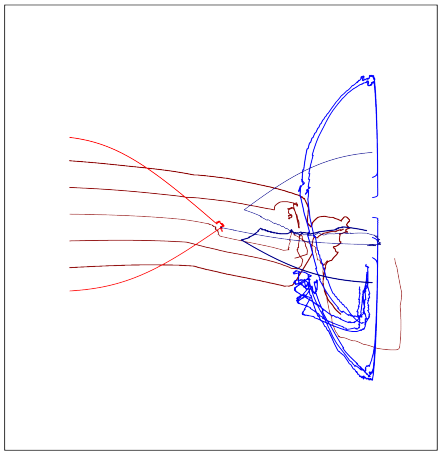


Fig. 4: Unit paths of an encounter between Zerg with 2 Zerglings (red), 5 Roaches (dark red) and Terran with 5 Marines (blue), 2 Marauders (dark blue). Winner: Terran.

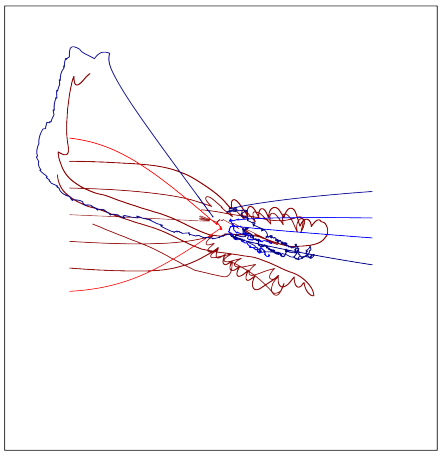


Fig. 5: Unit paths of an encounter between Zerg with 2 Zerglings (red), 5 Roaches (dark red) and Protoss with 2 Zealots (blue), 2 Stalkers (dark blue). Winner: Protoss.

V. Conclusion and Future Work

In this work a multi-objective genetic algorithm for simulating optimal fighting behavior in StarCraft II was presented. Its application aims to investigate the balancing of the game. By evaluating optimal build orders generated by the approach presented in [3] possible imbalances between the three races could be detected. However, as stable results could not be achieved in all cases, the approach needs further assessment, possibly requiring algorithmic improvements to achieve the mentioned goal. With the recent release of the second StarCraft II expansion pack Legacy of the Void™[13], an adaption of the simulation can be considered, which would

[13]Legacy of the Void is a trademark and StarCraft is a trademark or registered trademark of Blizzard Entertainment, Inc., in the U.S. and/or other countries.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.

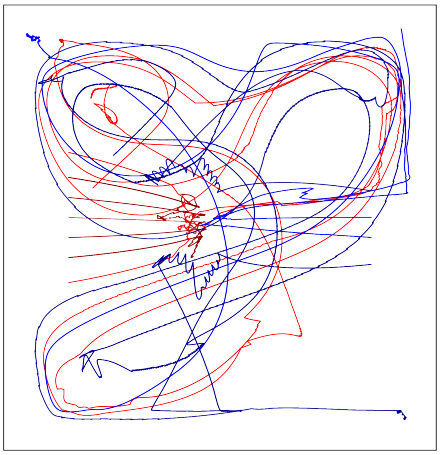


Fig. 6: Unit paths of an encounter between Terran with 5 Marines (red), 2 Marauders (dark red) and Protoss with 2 Zealots (blue), 2 Stalkers (dark blue). Winner: None (Time limit exceeded).

give eventual results more significance because the introduced changes could potentially lead to new imbalances. Although the presented approach was specifically designed to be applied to StarCraft II, which acts as a show case, its generality allows an easy adaption to similar games. As balancing in real time strategy games is generally hard to achieve, including time consuming testing phases involving human test players, its application can possibly shorten the game development cycle, allowing earlier releases. Moreover, because of the growing popularity of the eSports scene, to ensure equal winning chances for all participants of a competition, detecting and eliminating imbalances in games is of increasing importance.

VI. References

[1] S. Ontanón, G. Synnaeve, A. Uriarte, F. Richoux, D. Churchill, and M. Preuss, "A Survey of Real-Time Strategy Game AI Research and Competition in StarCraft," *Computational Intelligence and AI in Games, IEEE Transactions on*, vol. 5, no. 4, pp. 293–311, 2013.

[2] B. Gmeiner, G. Donnert, and H. Köstler, "Optimizing Opening Strategies in a Real-time Strategy Game by a Multi-objective Genetic Algorithm," in *Research and Development in Intelligent Systems XXIX*, pp. 361–374, Springer, 2012.

[3] H. Köstler and B. Gmeiner, "A Multi-Objective Genetic Algorithm for Build Order Optimization in StarCraft II," *KI-Künstliche Intelligenz*, vol. 27, no. 3, pp. 221–233, 2013.

[4] J. Hagelbäck and S. J. Johansson, "Using multi-agent potential fields in real-time strategy games," in *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 2*, pp. 631–638, International Foundation for Autonomous Agents and Multiagent Systems, 2008.

[5] G. Synnaeve and P. Bessiere, "A Bayesian model for RTS units control applied to StarCraft," in *Computational Intelligence and Games (CIG), 2011 IEEE Conference on*, pp. 190–196, IEEE, 2011.

[6] T. W. Sandberg and J. Togelius, *Evolutionary Multi-Agent potential field based AI approach for SSC scenarios in RTS games*. PhD thesis, 2011.

[7] J. Hagelbäck, "Potential-field based navigation in starcraft," in *Computational Intelligence and Games (CIG), 2012 IEEE Conference on*, pp. 388–393, IEEE, 2012.

[8] E. A. Rathe and J. B. Svendsen, "Micromanagement in Starcraft using potential fields tuned with a multi-objective genetic algorithm," 2012.

[9] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The international journal of robotics research*, vol. 5, no. 1, pp. 90–98, 1986.

[10] J. Barraquand, B. Langlois, and J.-C. Latombe, "Numerical potential field techniques for robot path planning," *Systems, Man and Cybernetics, IEEE Transactions on*, vol. 22, no. 2, pp. 224–241, 1992.

[11] J. Schmitt, S. Seufert, C. Zoubek, and H. Köstler, "Potential Field Based Unit Behavior Optimization for Balancing in StarCraft II," in *GECCO Companion '15: Proceedings of the Companion Publication of the 2015 on Genetic and Evolutionary Computation Conference*, (New York, NY, USA), pp. 1481–1482, ACM, 2015.

[12] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *Evolutionary Computation, IEEE Transactions on*, vol. 6, no. 2, pp. 182–197, 2002.

[13] K. A. De Jong and W. M. Spears, "A formal analysis of the role of multi-point crossover in genetic algorithms," *Annals of mathematics and Artificial intelligence*, vol. 5, no. 1, pp. 1–26, 1992.

[14] G. Syswerda, "Uniform Crossover in Genetic Algorithms," in *Proceedings of the 3rd International Conference on Genetic Algorithms*, (San Francisco, CA, USA), pp. 2–9, Morgan Kaufmann Publishers Inc., 1989.

[15] H. Mühlenbein and D. Schlierkamp-Voosen, "Predictive models for the breeder genetic algorithm i. continuous parameter optimization," *Evolutionary computation*, vol. 1, no. 1, pp. 25–49, 1993.

[16] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," *Complex Systems*, vol. 9, no. 3, pp. 1–15, 1994.

[17] K. Deb, K. Sindhya, and T. Okabe, "Self-adaptive simulated binary crossover for real-parameter optimization," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2007), UCL London*, pp. 1187–1194, Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2007), UCL London, 2007.

[18] K. Deb and D. Deb, "Analysing mutation schemes for real-parameter genetic algorithms," *International Journal of Artificial Intelligence and Soft Computing*, vol. 4, no. 1, pp. 1–28, 2014.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 28,2026 at 04:48:47 UTC from IEEE Xplore. Restrictions apply.